Here is the complete set of pseudocode logic for all 50 tasks:

1. Determine whether two given strings are anagrams of each otherPlaintextFUNCTION AreAnagrams(str1, str2):

```
  IF length(str1) != length(str2) THEN
    RETURN False
  END IF

  char_counts = empty Hash Map

  FOR EACH char IN str1 DO
    char_counts[char] = char_counts[char] + 1
  END FOR

  FOR EACH char IN str2 DO
    IF char NOT IN char_counts OR char_counts[char] == 0 THEN
      RETURN False
    END IF
    char_counts[char] = char_counts[char] - 1
  END FOR

  RETURN True
END FUNCTION
```

2. Convert a non-negative decimal integer into its binary string representationPlaintextFUNCTION DecimalToBinary(n):

```
  IF n == 0 THEN RETURN "0"

  binary_str = ""
  WHILE n > 0 DO
    remainder = n MOD 2
    binary_str = string(remainder) + binary_str
    n = n DIV 2
  END WHILE

  RETURN binary_str
```

```
END FUNCTION
```

3. Determine whether a given year satisfies the leap year rulesPlaintextFUNCTION IsLeapYear(year):

```
  IF (year MOD 4 == 0 AND year MOD 100 != 0) OR (year MOD 400 == 0) THEN
    RETURN True
  ELSE
    RETURN False
  END IF
END FUNCTION
```

4. Find the intersection of two sorted arrays and return the common elementsPlaintextFUNCTION FindIntersection(arr1, arr2):

```
  i = 0
  j = 0
  intersection = empty List

  WHILE i < length(arr1) AND j < length(arr2) DO
    IF arr1[i] == arr2[j] THEN
      IF length(intersection) == 0 OR intersection[last] != arr1[i] THEN
        APPEND arr1[i] TO intersection
      END IF
      i = i + 1
      j = j + 1
    ELSE IF arr1[i] < arr2[j] THEN
      i = i + 1
    ELSE
      j = j + 1
    END IF
  END WHILE

  RETURN intersection
END FUNCTION
```

5. Remove punctuation from a sentencePlaintextFUNCTION RemovePunctuation(sentence):

```
  punctuation_set = {'.', ',', '!', '?', ';', ':', '-', '\'', '"', '(', ')'}
  clean_sentence = ""
```

```
    FOR EACH char IN sentence DO
        IF char NOT IN punctuation_set THEN
            clean_sentence = clean_sentence + char
        END IF
    END FOR

    RETURN clean_sentence
END FUNCTION
```

6. Find the length of a given stringPlaintextFUNCTION GetStringLength(str):

```
    count = 0
    FOR EACH char IN str DO
        count = count + 1
    END FOR
    RETURN count
END FUNCTION
```

7. Split a sentence into individual words and store in a listPlaintextFUNCTION SplitSentence(sentence):

```
    words = empty List
    current_word = ""

    FOR EACH char IN sentence DO
        IF char == ' ' THEN
            IF current_word != "" THEN
                APPEND current_word TO words
                current_word = ""
            END IF
        ELSE
            current_word = current_word + char
        END IF
    END FOR

    IF current_word != "" THEN
```

```
        APPEND current_word TO words
    END IF

    RETURN words
END FUNCTION
```

8. Sort a list of words in alphabetical orderPlaintext

```
FUNCTION BubbleSortWords(words):
    n = length(words)
    FOR i FROM 0 TO n - 1 DO
        FOR j FROM 0 TO n - i - 2 DO
            IF words[j] > words[j + 1] THEN
                SWAP words[j] AND words[j + 1]
            END IF
        END FOR
    END FOR
    RETURN words
END FUNCTION
```

9. Find the exact position of T in an unsorted list, or report non-existencePlaintext

```
FUNCTION LinearSearch(arr, T):
    FOR index FROM 0 TO length(arr) - 1 DO
        IF arr[index] == T THEN
            RETURN "Target found at index: " + index
        END IF
    END FOR
    RETURN "Target does not exist in the list."
END FUNCTION
```

10. Scan text and count how many times each vowel (a, e, i, o, u) appearsPlaintext

```
FUNCTION CountVowels(text):
    vowel_counts = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

    FOR EACH char IN lowercase(text) DO
        IF char IN vowel_counts THEN
            vowel_counts[char] = vowel_counts[char] + 1
        END IF
    END FOR
```

```
    RETURN vowel_counts
END FUNCTION
```

11. Remove duplicate IP addresses and return unique IPsPlaintextFUNCTION GetUniqueIPs(ip_list):

```
    unique_set = empty Set
    FOR EACH ip IN ip_list DO
        ADD ip TO unique_set
    END FOR
    RETURN ConvertSetToList(unique_set)
END FUNCTION
```

12. Calculate sum of the main diagonal in an $N \times N$ matrixPlaintextFUNCTION SumMainDiagonal(matrix, N):

```
    diagonal_sum = 0
    FOR i FROM 0 TO N - 1 DO
        diagonal_sum = diagonal_sum + matrix[i][i]
    END FOR
    RETURN diagonal_sum
END FUNCTION
```

13. Reverse a string without using built-in functionsPlaintextFUNCTION ReverseString(str):

```
    reversed_str = ""
    FOR i FROM length(str) - 1 DOWN TO 0 DO
        reversed_str = reversed_str + str[i]
    END FOR
    RETURN reversed_str
END FUNCTION
```

14. Find the sum of all even numbers in a listPlaintextFUNCTION SumEvenNumbers(numbers):

```
    total_sum = 0
    FOR EACH num IN numbers DO
        IF num MOD 2 == 0 THEN
            total_sum = total_sum + num
        END IF
    END FOR
    RETURN total_sum
```

END FUNCTION

15. Find the second largest element in an arrayPlaintextFUNCTION FindSecondLargest(arr):

```
  IF length(arr) < 2 THEN RETURN NULL

  largest = -INFINITY
  second_largest = -INFINITY

  FOR EACH num IN arr DO
    IF num > largest THEN
      second_largest = largest
      largest = num
    ELSE IF num > second_largest AND num != largest THEN
      second_largest = num
    END IF
  END FOR

  RETURN second_largest
END FUNCTION
```

16. Find GCD of two numbers using Euclidean algorithmPlaintextFUNCTION FindGCD(a, b):

```
  WHILE b != 0 DO
    temp = b
    b = a MOD b
    a = temp
  END WHILE
  RETURN a
END FUNCTION
```

17. Compute the sum and average of a list of numbersPlaintextFUNCTION ComputeSumAndAverage(numbers):

```
  IF length(numbers) == 0 THEN RETURN (0, 0)

  total_sum = 0
  FOR EACH num IN numbers DO
    total_sum = total_sum + num
```

```
    END FOR

    average = total_sum / length(numbers)
    RETURN (total_sum, average)
END FUNCTION
```

18. Count the frequency of each character in a stringPlaintextFUNCTION CharacterFrequency(str):

```
    freq_map = empty Hash Map
    FOR EACH char IN str DO
        IF char IN freq_map THEN
            freq_map[char] = freq_map[char] + 1
        ELSE
            freq_map[char] = 1
        END IF
    END FOR
    RETURN freq_map
END FUNCTION
```

19. Swap two variables without using a temporary third variablePlaintextPROCEDURE SwapVariables(a, b):

```
    a = a + b
    b = a - b
    a = a - b
END PROCEDURE
```

20. Determine all divisors of a given positive integerPlaintextFUNCTION FindDivisors(n):

```
    divisors = empty List
    FOR i FROM 1 TO SQUARE_ROOT(n) DO
        IF n MOD i == 0 THEN
            APPEND i TO divisors
            IF i != (n DIV i) THEN
                APPEND (n DIV i) TO divisors
            END IF
        END IF
    END FOR
    RETURN divisors
```

END FUNCTION

21. Create a process that determines whether a number is even or oddPlaintextFUNCTION CheckEvenOrOdd(num):

IF num MOD 2 == 0 THEN

RETURN "Even"

ELSE

RETURN "Odd"

END IF

END FUNCTION

22. Design a procedure that computes the factorial of a positive integerPlaintextFUNCTION ComputeFactorial(n):

IF n < 0 THEN RETURN NULL

result = 1

FOR i FROM 1 TO n DO

result = result * i

END FOR

RETURN result

END FUNCTION

23. Convert numerical scores into grade categories (Pass, Merit, Distinction)PlaintextFUNCTION ConvertScoreToGrade(score):

IF score >= 70 THEN

RETURN "Distinction"

ELSE IF score >= 60 THEN

RETURN "Merit"

ELSE IF score >= 40 THEN

RETURN "Pass"

ELSE

RETURN "Fail"

END IF

END FUNCTION

24. Validate password (8+ chars, 1 number, 1 uppercase letter)PlaintextFUNCTION ValidatePassword(password):

IF length(password) < 8 THEN RETURN False

has_digit = False

has_uppercase = False

```
    FOR EACH char IN password DO
        IF IS_DIGIT(char) THEN has_digit = True
        IF IS_UPPERCASE(char) THEN has_uppercase = True
    END FOR

    RETURN (has_digit AND has_uppercase)
END FUNCTION
```

25. Find length of longest substring without repeating charactersPlaintextFUNCTION LongestUniqueSubstringLength(str):

```
    char_map = empty Hash Map
    max_len = 0
    start = 0

    FOR end FROM 0 TO length(str) - 1 DO
        char = str[end]
        IF char IN char_map AND char_map[char] >= start THEN
            start = char_map[char] + 1
        END IF
        char_map[char] = end
        max_len = MAX(max_len, end - start + 1)
    END FOR

    RETURN max_len
END FUNCTION
```

26. Perform level-order traversal on a binary treePlaintextFUNCTION LevelOrderTraversal(root):

```
    IF root IS NULL THEN RETURN empty List

    result = empty List
    queue = empty Queue
    ENQUEUE root TO queue

    WHILE queue IS NOT EMPTY DO
```

```
    level_size = queue.size()
    current_level = empty List

    FOR i FROM 1 TO level_size DO
      node = DEQUEUE from queue
      APPEND node.value TO current_level

      IF node.left IS NOT NULL THEN ENQUEUE node.left TO queue
      IF node.right IS NOT NULL THEN ENQUEUE node.right TO queue
    END FOR

    APPEND current_level TO result
  END WHILE

  RETURN result
END FUNCTION
```

27. Rotate a square 2D matrix 90 degrees clockwise in-placePlaintextPROCEDURE RotateMatrixClockwise(matrix, N):

```
  // Step 1: Transpose matrix
  FOR i FROM 0 TO N - 1 DO
    FOR j FROM i TO N - 1 DO
      SWAP matrix[i][j] AND matrix[j][i]
    END FOR
  END FOR

  // Step 2: Reverse each row
  FOR i FROM 0 TO N - 1 DO
    left = 0
    right = N - 1
    WHILE left < right DO
      SWAP matrix[i][left] AND matrix[i][right]
      left = left + 1
      right = right - 1
```

```
    END WHILE
  END FOR
END PROCEDURE
```

28. Sliding window maximumPlaintextFUNCTION SlidingWindowMaximum(arr, k):

```
  result = empty List
  deque = empty DoubleEndedQueue // Stores indices

  FOR i FROM 0 TO length(arr) - 1 DO
    IF deque IS NOT EMPTY AND deque.front() <= i - k THEN
      REMOVE FRONT FROM deque
    END IF

    WHILE deque IS NOT EMPTY AND arr[deque.back()] <= arr[i] DO
      REMOVE BACK FROM deque
    END WHILE

    ADD BACK i TO deque

    IF i >= k - 1 THEN
      APPEND arr[deque.front()] TO result
    END IF
  END FOR

  RETURN result
END FUNCTION
```

29. Build a text classification model processPlaintextPROCEDURE TextClassificationPipeline(corpus, labels):

```
  processed_corpus = PreprocessText(corpus)
  X_train, X_test, y_train, y_test = SplitData(processed_corpus, labels)

  vectorizer = TrainTFIDFVectorizer(X_train)
  X_train_vec = vectorizer.transform(X_train)
```

```
    model = TrainClassifierModel(X_train_vec, y_train)

    X_test_vec = vectorizer.transform(X_test)
    predictions = model.predict(X_test_vec)

    EvaluateModel(predictions, y_test)
END PROCEDURE
```

30. Topic modelling system (LDA approach)Plaintext

```
PROCEDURE TopicModellingSystem(corpus, num_topics):
    cleaned_corpus = RemoveStopwordsAndPunctuation(corpus)
    tokenized_docs = TokenizeAndStem(cleaned_corpus)

    doc_term_matrix = CreateDocumentTermMatrix(tokenized_docs)
    lda_model = InitializeLDAModel(num_topics)

    FitLDAModel(lda_model, doc_term_matrix)

    FOR EACH topic IN lda_model.get_topics() DO
        PRINT ExtractTopWords(topic, top_n=10)
    END FOR
END PROCEDURE
```

31. Co-reference resolution systemPlaintext

```
FUNCTION CoreferenceResolution(text):
    tokens, pos_tags = TokenizeAndTag(text)
    entities = ExtractNamedEntities(tokens, pos_tags)
    pronouns = ExtractPronouns(tokens, pos_tags)

    resolved_text = text
    FOR EACH pronoun IN pronouns DO
        antecedent = FindMostCompatibleEntity(pronoun, entities, search_window=2_sentences)
        IF antecedent IS NOT NULL THEN
            ReplaceInText(resolved_text, pronoun, antecedent.name)
        END IF
    END FOR
```

```
  RETURN resolved_text
END FUNCTION
```

32. Question answering system (Retrieval-Augmented)

```
FUNCTION AnswerQuestion(user_query, context_text):
  text_chunks = ChunkDocument(context_text, chunk_size=500)
  chunk_embeddings = Embed(text_chunks)
  query_embedding = Embed(user_query)

  best_chunk = FindHighestCosineSimilarity(query_embedding, chunk_embeddings)
  answer = ExtractOrGenerateAnswer(user_query, best_chunk)

  RETURN answer
END FUNCTION
```

33. Square root using binary search (rounded down)

```
FUNCTION IntegerSquareRoot(n):
  IF n < 2 THEN RETURN n

  low = 1
  high = n DIV 2
  ans = 0

  WHILE low <= high DO
    mid = low + (high - low) DIV 2
    IF mid * mid == n THEN
      RETURN mid
    ELSE IF mid * mid < n THEN
      ans = mid
      low = mid + 1
    ELSE
      high = mid - 1
    END IF
  END WHILE
```

```
    RETURN ans
END FUNCTION
```

34. Unique common elements from two unsorted listsPlaintextFUNCTION FindUniqueCommonElements(list1, list2):

```
    set1 = ConvertToSet(list1)
    result_set = empty Set

    FOR EACH item IN list2 DO
        IF item IN set1 THEN
            ADD item TO result_set
        END IF
    END FOR

    RETURN ConvertSetToList(result_set)
END FUNCTION
```

35. Least Recently Used (LRU) CachePlaintextCLASS LRUCache:

```
    CONSTRUCTOR(capacity):
        self.capacity = capacity
        self.map = empty Hash Map
        self.doubly_linked_list = empty DoublyLinkedList

    FUNCTION Get(key):
        IF key NOT IN self.map THEN RETURN -1
        node = self.map[key]
        self.doubly_linked_list.moveToHead(node)
        RETURN node.value

    PROCEDURE Put(key, value):
        IF key IN self.map THEN
            node = self.map[key]
            node.value = value
            self.doubly_linked_list.moveToHead(node)
        ELSE
```

```
    IF self.map.size() >= self.capacity THEN
      tail_node = self.doubly_linked_list.removeTail()
      REMOVE tail_node.key FROM self.map
    END IF
    new_node = CreateNode(key, value)
    self.doubly_linked_list.addToHead(new_node)
    self.map[key] = new_node
  END IF
END PROCEDURE
```

36. Search in a rotated sorted array (Logarithmic time)PlaintextFUNCTION SearchRotatedArray(arr, target):

```
low = 0
high = length(arr) - 1

WHILE low <= high DO
  mid = low + (high - low) DIV 2
  IF arr[mid] == target THEN RETURN mid

  // Left side is sorted
  IF arr[low] <= arr[mid] THEN
    IF arr[low] <= target AND target < arr[mid] THEN
      high = mid - 1
    ELSE
      low = mid + 1
    END IF
  // Right side is sorted
  ELSE
    IF arr[mid] < target AND target <= arr[high] THEN
      low = mid + 1
    ELSE
      high = mid - 1
    END IF
  END IF
```

```
    END WHILE

    RETURN -1
END FUNCTION
```

37. Implement a Stack using an ArrayPlaintextCLASS Stack:

```
    CONSTRUCTOR(capacity):
        self.arr = Array of size capacity
        self.top_index = -1

    PROCEDURE Push(item):
        IF self.top_index == length(self.arr) - 1 THEN RETURN "Overflow Error"
        self.top_index = self.top_index + 1
        self.arr[self.top_index] = item

    FUNCTION Pop():
        IF self.top_index == -1 THEN RETURN "Underflow Error"
        item = self.arr[self.top_index]
        self.top_index = self.top_index - 1
        RETURN item

    FUNCTION Peek():
        IF self.top_index == -1 THEN RETURN "Stack Empty"
        RETURN self.arr[self.top_index]
```

38. Find the first non-repeating character in a stringPlaintextFUNCTION FirstNonRepeatingChar(str):

```
    counts = empty Hash Map

    FOR EACH char IN str DO
        counts[char] = counts[char] + 1
    END FOR

    FOR EACH char IN str DO
        IF counts[char] == 1 THEN
```

```
        RETURN char
      END IF
    END FOR

    RETURN NULL
END FUNCTION
```

39. Detect cycle in a singly linked list (Floyd’s algorithm)PlaintextFUNCTION HasCycle(head):

```
    IF head IS NULL OR head.next IS NULL THEN RETURN False

    slow = head
    fast = head

    WHILE fast IS NOT NULL AND fast.next IS NOT NULL DO
      slow = slow.next
      fast = fast.next.next

      IF slow == fast THEN
        RETURN True
      END IF
    END WHILE

    RETURN False
END FUNCTION
```

40. Implement a Queue using two StacksPlaintextCLASS QueueUsingStacks:

```
    CONSTRUCTOR():
      self.stack_in = Stack()
      self.stack_out = Stack()

    PROCEDURE Enqueue(item):
      self.stack_in.push(item)

    FUNCTION Dequeue():
```

```
    IF self.stack_out.isEmpty() THEN
      WHILE NOT self.stack_in.isEmpty() DO
        self.stack_out.push(self.stack_in.pop())
      END WHILE
    END IF

    IF self.stack_out.isEmpty() THEN RETURN "Queue Empty"
    RETURN self.stack_out.pop()
```

41. Solve the N Queens problemPlaintext

```
FUNCTION SolveNQueens(N):
  board = N x N grid initialized to 0
  solutions = empty List

  PROCEDURE Backtrack(row, cols, diag1, diag2):
    IF row == N THEN
      APPEND Copy(board) TO solutions
      RETURN
    END IF

    FOR col FROM 0 TO N - 1 DO
      IF col IN cols OR (row - col) IN diag1 OR (row + col) IN diag2 THEN
        CONTINUE
      END IF

      board[row][col] = 1
      Backtrack(row + 1, cols + {col}, diag1 + {row - col}, diag2 + {row + col})
      board[row][col] = 0 // Backtrack
    END FOR
  END PROCEDURE

  Backtrack(0, empty Set, empty Set, empty Set)
  RETURN solutions
END FUNCTION
```

42. Find all pairs in an array that sum to a target valuePlaintextFUNCTION FindPairsWithSum(arr, target):

```
  seen = empty Set
  pairs = empty Set

  FOR EACH num IN arr DO
    complement = target - num
    IF complement IN seen THEN
      ADD (MIN(num, complement), MAX(num, complement)) TO pairs
    END IF
    ADD num TO seen
  END FOR

  RETURN pairs
END FUNCTION
```

43. Solve Tower of Hanoi for N discsPlaintextPROCEDURE TowerOfHanoi(N, source, auxiliary, target):

```
  IF N == 1 THEN
    PRINT "Move disc 1 from " + source + " to " + target
    RETURN
  END IF

  TowerOfHanoi(N - 1, source, target, auxiliary)
  PRINT "Move disc " + N + " from " + source + " to " + target
  TowerOfHanoi(N - 1, auxiliary, source, target)
END PROCEDURE
```

44. Find contiguous subarray with maximum sum (Kadane’s algorithm)PlaintextFUNCTION MaxSubarraySum(arr):

```
  max_so_far = arr[0]
  current_max = arr[0]

  FOR i FROM 1 TO length(arr) - 1 DO
    current_max = MAX(arr[i], current_max + arr[i])
    max_so_far = MAX(max_so_far, current_max)
  END FOR
```

```
    RETURN max_so_far
END FUNCTION
```

45. Add two matrices of equal sizePlaintextFUNCTION AddMatrices(A, B, rows, cols):

```
    C = rows x cols matrix filled with 0
    FOR i FROM 0 TO rows - 1 DO
        FOR j FROM 0 TO cols - 1 DO
            C[i][j] = A[i][j] + B[i][j]
        END FOR
    END FOR
    RETURN C
END FUNCTION
```

46. Multiply two matricesPlaintextFUNCTION MultiplyMatrices(A, B, r1, c1, r2, c2):

```
    IF c1 != r2 THEN RETURN "Invalid matrix dimensions"

    C = r1 x c2 matrix filled with 0
    FOR i FROM 0 TO r1 - 1 DO
        FOR j FROM 0 TO c2 - 1 DO
            FOR k FROM 0 TO c1 - 1 DO
                C[i][j] = C[i][j] + (A[i][k] * B[k][j])
            END FOR
        END FOR
    END FOR
    RETURN C
END FUNCTION
```

47. Inventory quantities update after purchases and deliveriesPlaintextPROCEDURE UpdateInventory(inventory_map, transactions):

```
    FOR EACH item IN transactions DO
        IF item.type == "PURCHASE" THEN
            inventory_map[item.id] = inventory_map[item.id] - item.quantity
        ELSE IF item.type == "DELIVERY" THEN
            inventory_map[item.id] = inventory_map[item.id] + item.quantity
```

```
    END IF
  END FOR
END PROCEDURE
```

48. Recursive search for file type in nested foldersPlaintextFUNCTION SearchFiles(directory, target_extension):

```
  matching_files = empty List

  FOR EACH item IN directory.getItems() DO
    IF item.isFolder() THEN
      matching_files.extend(SearchFiles(item, target_extension))
    ELSE IF item.getExtension() == target_extension THEN
      APPEND item.path TO matching_files
    END IF
  END FOR

  RETURN matching_files
END FUNCTION
```

49. Maze pathfinding (BFS for shortest path)PlaintextFUNCTION SolveMaze(maze, start, end):

```
  queue = empty Queue
  visited = empty Set
  parent = empty Hash Map

  ENQUEUE start TO queue
  ADD start TO visited

  WHILE queue IS NOT EMPTY DO
    curr = DEQUEUE from queue
    IF curr == end THEN RETURN ReconstructPath(parent, start, end)

    FOR EACH neighbor IN GetValidNeighbors(maze, curr) DO
      IF neighbor NOT IN visited THEN
        ADD neighbor TO visited
        parent[neighbor] = curr
```

```
            ENQUEUE neighbor TO queue
        END IF
    END FOR
  END WHILE

  RETURN "No path found"
END FUNCTION
```

50. Autonomous drone swarm collision avoidance

```
PROCEDURE CoordinateDroneMovement(drones, safety_radius):
  FOR EACH drone IN drones DO
    desired_vector = CalculateTargetVector(drone, drone.destination)
    repulsion_vector = Vector(0, 0, 0)

    FOR EACH other_drone IN drones DO
      IF drone.id != other_drone.id THEN
        dist = Distance(drone.position, other_drone.position)
        IF dist < safety_radius THEN
          repulsion = Direction(other_drone.position -> drone.position) / dist
          repulsion_vector = repulsion_vector + repulsion
        END IF
      END IF
    END FOR

    adjusted_velocity = desired_vector + (repulsion_vector * AVOIDANCE_WEIGHT)
    drone.setVelocity(LimitSpeed(adjusted_velocity))
  END FOR
END PROCEDURE
```